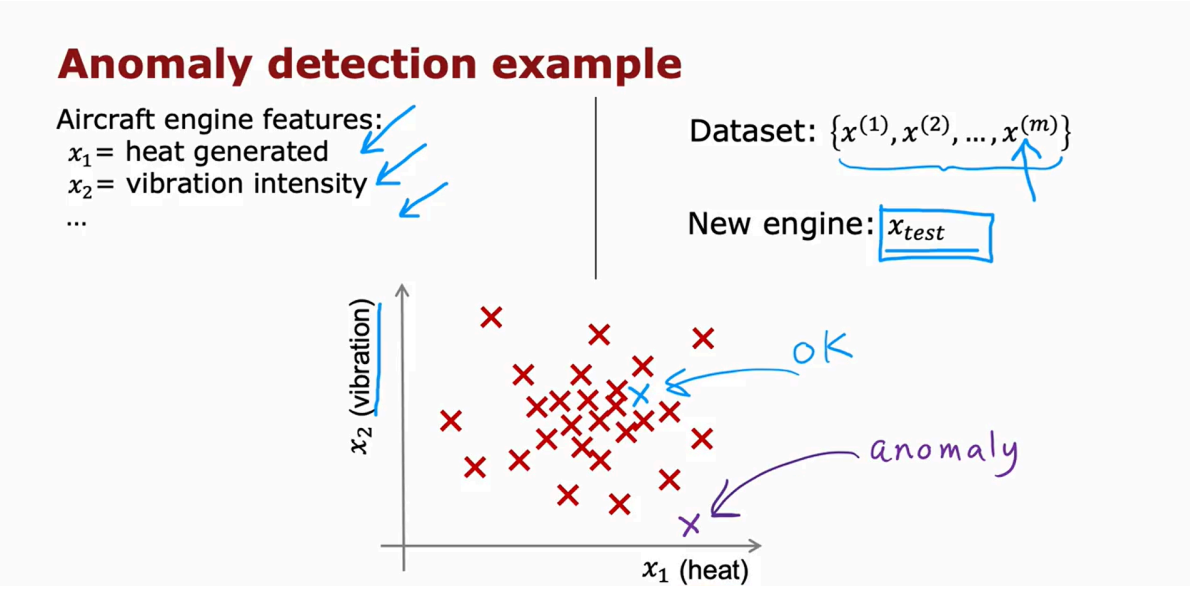
Anomaly detection

Finding unusual events

Example



Some of my friends were working on using anomaly detection to detect possible problems with aircraft engines that were being manufactured. When a company makes an aircraft engine, you really want that aircraft engine to be reliable and function well because an aircraft engine failure has very negative consequences.

So some of my friends were using anomaly detection to check if an aircraft engine after it was manufactured seemed anomalous or if there seemed to be anything wrong with it.

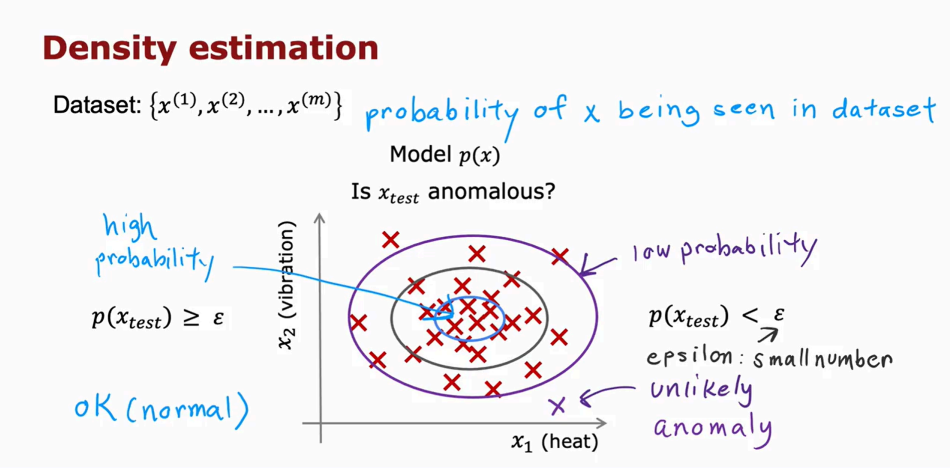
Here's the idea, after an aircraft engine rolls off the assembly line, you can compute a number of different features of the aircraft engine. So, say feature x1 measures the heat generated by the engine. Feature x2 measures the vibration intensity and so on and so forth for additional features as well. But to simplify the slide a bit, I'm going to use just two features x1 and x2 corresponding to the heat and the vibrations of the engine.

Now, it turns out that aircraft engine manufacturers don't make that many bad engines. And so the easier type of data to collect would be if you have

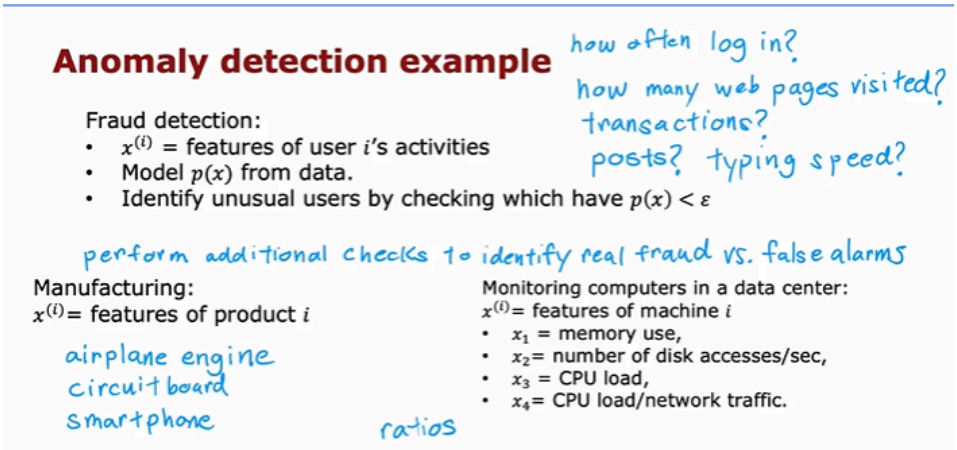
manufactured m aircraft engines to collect the features x1 and x2 about how these m engines (x_1 to x_m) behave and probably most of them are just fine that normal engines rather than ones with a defect or flaw in them.

Density estimation

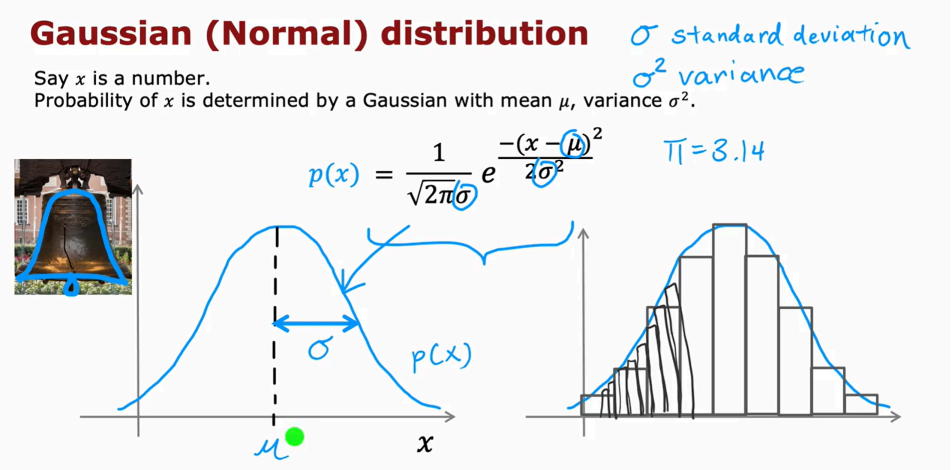
The most common way to carry out anomaly detection is through a technique called density estimation. And what that means is, when you're given your training sets of these m examples, the first thing you do is build a model for the probability of x.



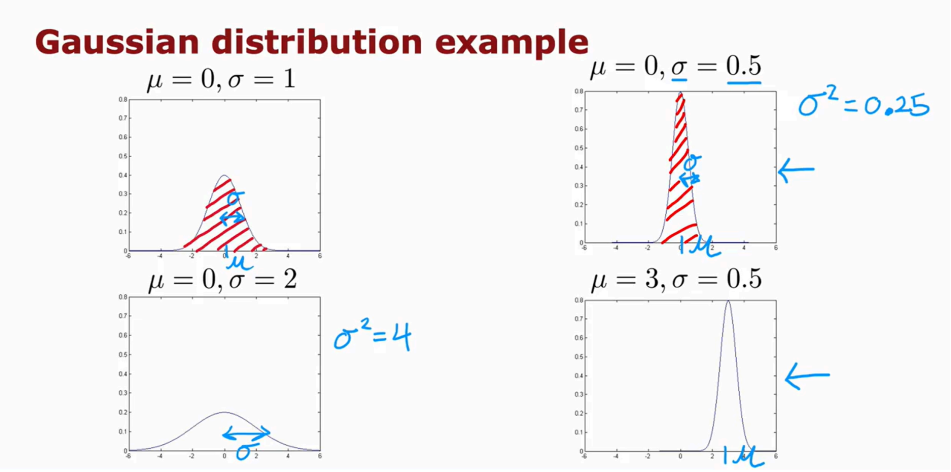
And this type of fraud detection is used both to find fake accounts and this type of algorithm is also used frequently to try to identify financial fraud such as if there's a very unusual pattern of purchases. Then that may be something well worth a security team taking a more careful look at.



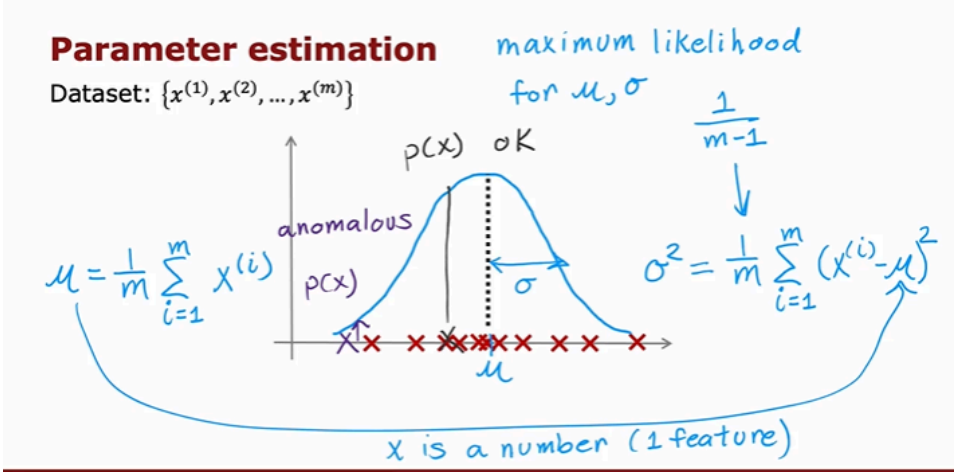
Gaussian (normal) distribution



For any given value of μ and σ, if you were to plot this function as a function of x (p(x)), you get this type of bell-shaped curve that is centered at μ, and with the width of this bell-shaped curve being determined by the parameter σ.



You may have heard that probabilities always have to sum up to one, so that's why the area under the curve is always equal to one (= 1), which is why when the Gaussian distribution becomes skinnier, it has to become taller as well.



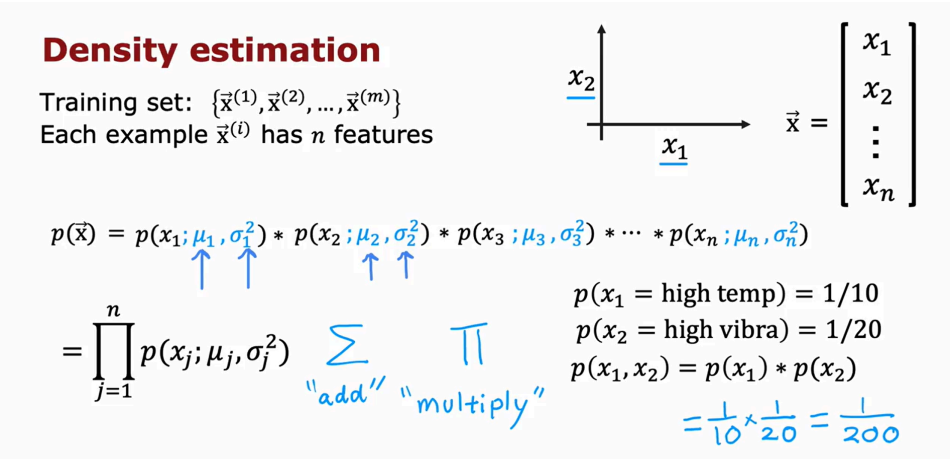
If you've taken an advanced statistics class, you may have heard that these formulas for μ and σ squared are technically called the maximum likelihood estimates for μ and σ.

Some statistics classes will tell you to use the formula $\frac{1}{m-1}$ instead of $\frac{1}{m}$. In practice, using $\frac{1}{m}$ or $\frac{1}{m-1}$ makes very little difference. I always use $\frac{1}{m}$, but just some other properties of $\frac{1}{m-1}$ that some statisticians prefer.

You've now seen how the Gaussian distribution works. If x is a single number, this corresponds to if, say you had just one feature for your anomaly detection problem.

But for practical anomaly detection applications, you will have many features, two or three or some even larger number n of features. Let's take what you saw for a single Gaussian and use it to build a more sophisticated anomaly detection algorithm.

Anomaly detection algorithm



In the case of the airplane engine example, we had two features corresponding to the heat and the vibrations. And so, each of these $\vec{x}^{(i)}$ would be a two dimensional vector and n would be equal to 2. But for many practical applications n can be much larger and you might do this with dozens or even hundreds of features.

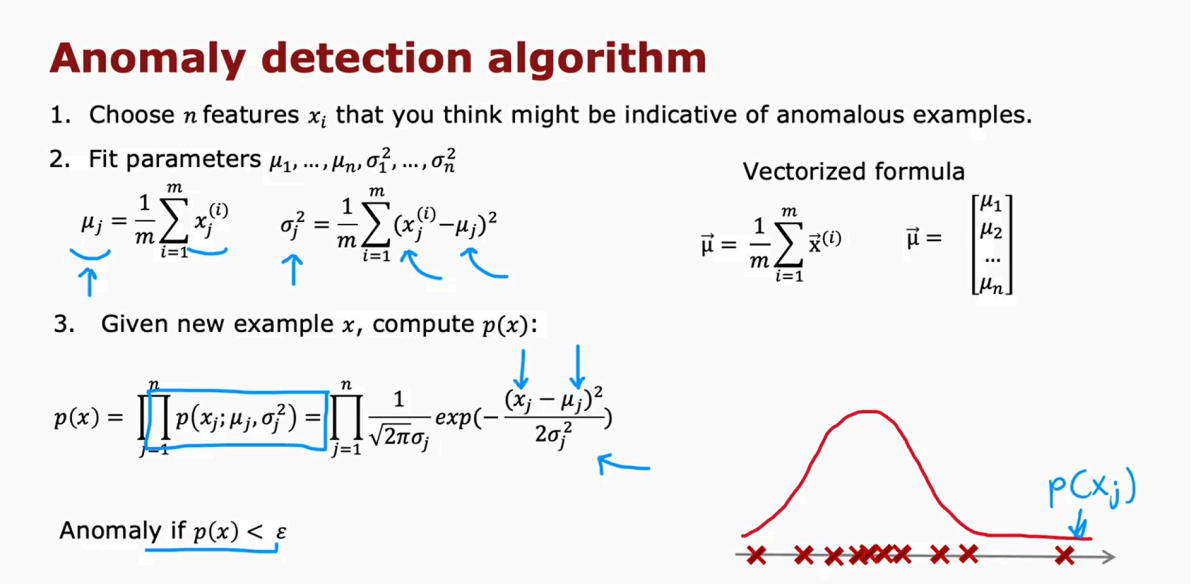
Given this training set, what we would like to do is to carry out density estimation and all that means is, we will build a model or estimate the probability for $p(\vec{x})$.

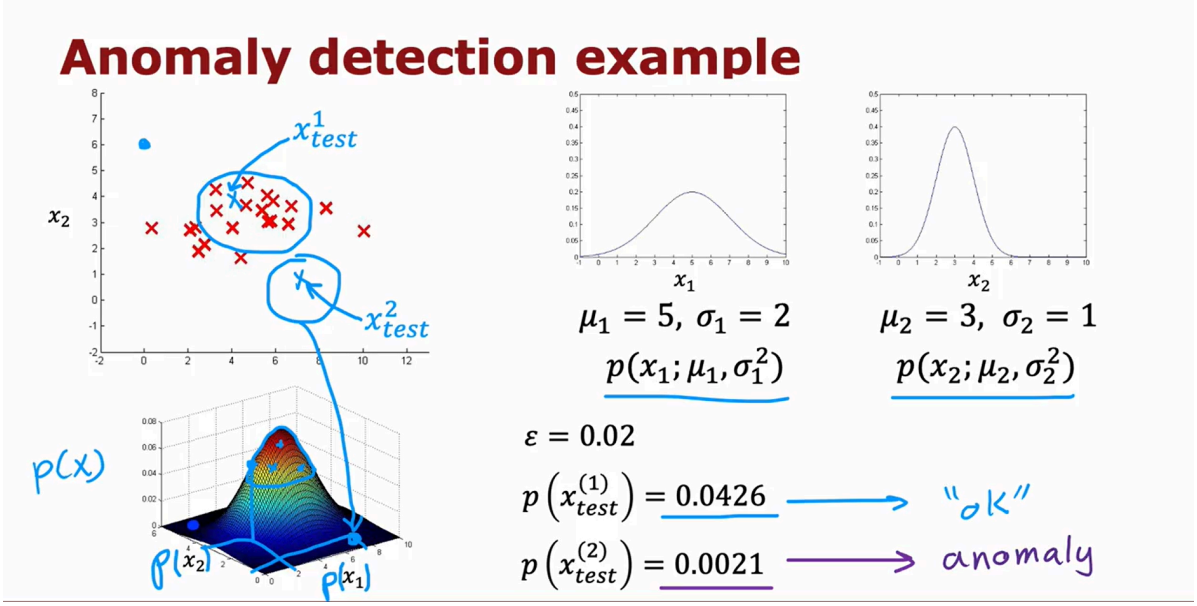
If you've taken an advanced class in probably in statistics before, you may recognize that this equation $p(\vec{x}) = ...$ corresponds to assuming that the features x_1, x_2 and so on up to x_n are statistically independent.

But it turns out this algorithm often works fine even that the features are not actually statistically independent. But if you don't understand what I just said, don't worry about it. Understanding statistical independence is not needed to fully complete this class and also, be able to very effectively use anomaly detection algorithm.

And in order to model the probability of x_1, say the heat feature in this example we're going to have two parameters, μ_1 and σ_1 or $\sigma_1^2 = 1$. And what that means is we're going to estimate, the mean of the feature x_1 and also the variance of feature x_1 and that will be μ_1 and σ_1.

To model $p(x_2; ...)$, x_2 is a totally different feature measuring the vibrations of the airplane engine. We're going to have two different parameters, which I'm going to write as μ_2, σ_2^2. And it turns out this μ_2 will correspond to the mean or the average of the vibration feature and the variance (σ_2^2) of the vibration feature and so on. If you have additional features μ_3 and σ_3^2 up through μ_n and σ_n^2.



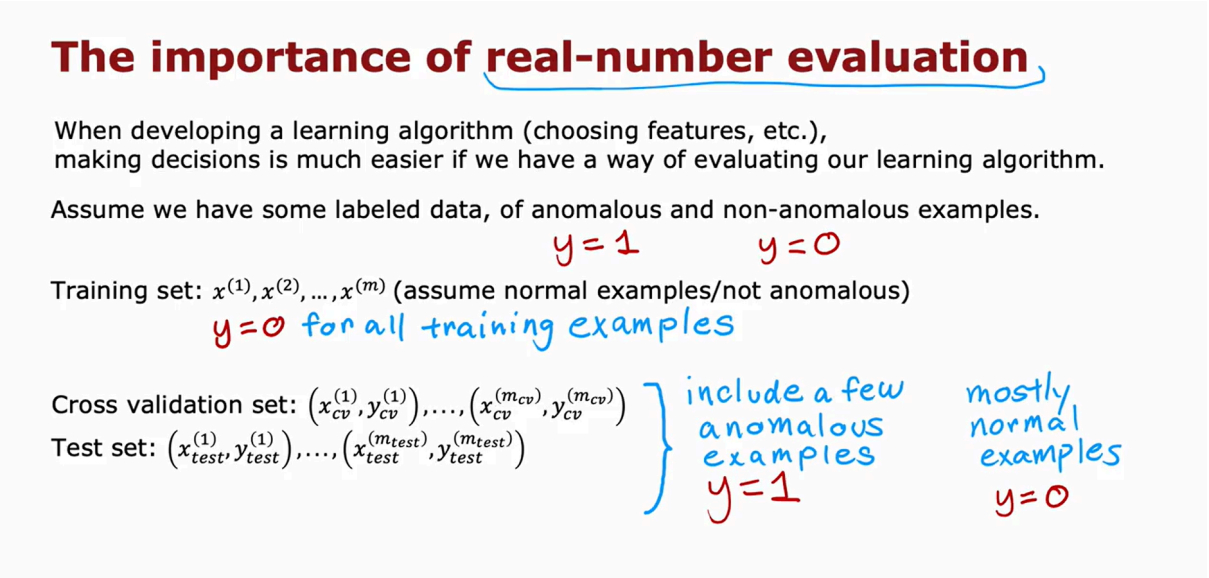


If you were to actually multiply $p(x_1)$ and $p(x_2)$, then you end up with this three Dimension surface plot for $p(X)$ where any point, the height of this is the product of $p(x_1) \cdot p(x_2)$.

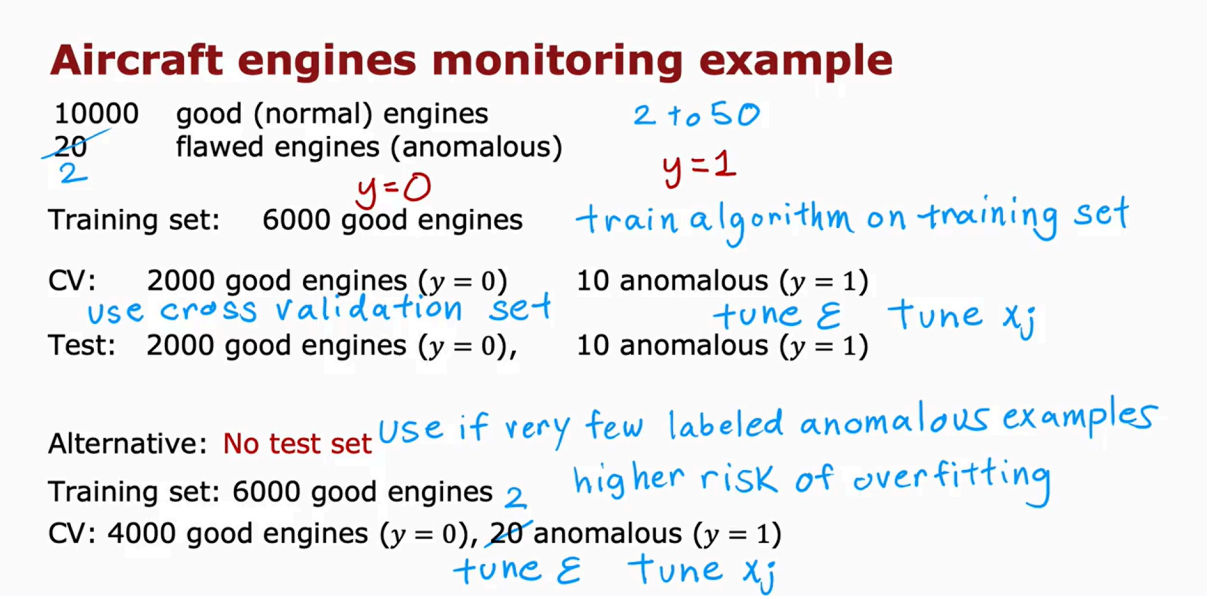
So you've seen the process of how to build an anomaly detection system. But how do you choose the parameter epsilon? And how do you know if your anomaly detection system is working well in the next video, let's dive a little bit more deeply into the process of developing and evaluating the performance of an anomaly detection system.

Developing and evaluating an anomaly detection system

One of the key ideas will be that if you can have a way to evaluate a system, even as it's being developed, you'll be able to make decisions and change the system and improve it much more quickly.



This is similar notation as you had seen in the second course of this specialization. As similarly, have a test set of some number of examples where both the cross validation and test sets hopefully includes a few anomalous examples. In other words, the cross validation and test sets will have a few examples of y equals 1, but also a lot of examples where y = 0



Let's illustrate this with the aircraft engine example. Let's say you have been manufacturing aircraft engines for years and so you've collected data from 10,000 good or normal engines, but over the years you had also collected data from 20 flawed or anomalous engines.

Usually the number of anomalous engines, that is y = 1, will be much smaller. It will not be a typical to apply this type of algorithm with anywhere from, say, 2-50 known anomalies. We're going to take this dataset and break it up into a training set, a cross validation set, and the test set.

Here's one example. I'm going to put 6,000 good engines into the training set. Again, if there are couple of anomalous engines that got slipped into this set is actually okay, I wouldn't worry too much about that. Then let's put 2,000 good engines and 10 of the known anomalies into the cross-validation set, and a separate 2,000 good and 10 anomalous engines into the test set. What you can do then is train the algorithm on the training set, fit the Gaussian distributions to these 6,000 examples and then on the cross-validation set, you can see how many of the anomalous engines it correctly flags.

For example, you could use the cross validation set to tune (điều chỉnh) the parameter epsilon and set it higher or lower depending on whether the algorithm seems to be reliably detecting these 10 anomalies without taking too many of these 2,000 good engines and flagging them as anomalies. After you have tuned the parameter epsilon and maybe also added or subtracted or tuned to features x_j you can then take the algorithm and evaluate it on your test set to see how many of these 10 anomalous engines it finds, as well as how many mistakes it makes by flagging the good engines as anomalous ones.

Notice that this is still primarily an unsupervised learning algorithm because the training sets really has no labels or they all have labels that we're assuming to be y equals 0 and so we learned from the training set by fitting the Gaussian distributions as you saw in the previous video. But it turns out if you're building a practical anomaly detection system, having a small number of anomalies to use to evaluate the algorithm that your cross validation and test sets is very helpful for tuning the algorithm.

Because the number of flawed engines is so small there's one other alternative that I often see people use for anomaly detection, which is to not use a test set, like to have just a training set and a cross-validation set.

In this example, you will set train on 6,000 good engines, but take the remainder of the data, the 4,000 remaining good engines as well as all the anomalies, and put them in the cross validation set. You would then tune the parameters Epsilon and add or subtract features x_j to try to get it to do as well as possible as evaluated on the cross validation set. If you have very few flawed engines, so if you had only 2 flawed engines, then this really makes sense to put all of that in the cross validation set.

You just don't have enough data to create a totally separate test set that is distinct from your cross-validation set. The downside of this alternative here is that after you've tuned your algorithm, you don't have a fair way to tell how well this will actually do on future examples because you don't have the test set. But when your dataset is small, especially when the number of anomalies you have, your dataset is small, this might be the best alternative you have. I see this done quite often as well when you just don't have enough data to create a separate test set.

If this is the case, just be aware that there's a higher risk that you will have over-fit some of your decisions around Epsilon and choice of features and so on to the cross-validation set and so its performance on real data in the future may not be as good as you were expecting.

Algorithm evaluation

course 2 week 3
skewed datasets

Fit model $p(x)$ on training set $x^{(1)}, x^{(2)}, \ldots, x^{(m)}$
On a cross validation/test example x, predict

$$y = \begin{cases} 1 & if\ p(x) < \varepsilon\ (anomaly) \\ 0 & if\ p(x) \geq \varepsilon\ (normal) \end{cases}$$

10
2000

Possible evaluation metrics:

- True positive, false positive, false negative, true negative
- Precision/Recall
- F_1-score

Use cross validation set to choose parameter ε

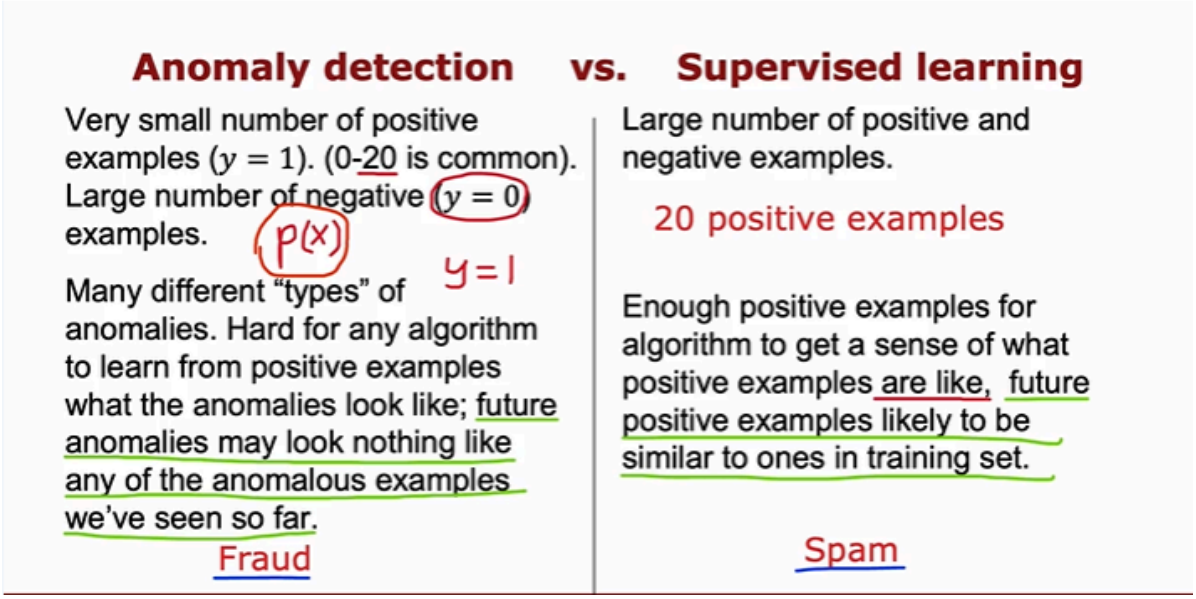
In the third week of the second course, we had had a couple of optional videos on how to handle highly skewed data distributions where the number of positive examples, y equals 1, can be much smaller than the number of negative examples where y equals 0.

This is the case as well for many anomaly detection in the applications where the number of anomalies in your cross-validation set is much smaller. In our previous example, we had maybe 10 positive examples and 2,000 negative examples because we had 10 anomalies and 2,000 normal examples. If you saw those optional videos, you may recall that we saw it can be useful to compute things like the true positive, false positive, false negative, and true negative rates.

Also compute precision recall or F_1 score and that these are alternative metrics and classification accuracy that could work better when your data distribution is very skewed. If you saw that video, you might consider applying those types of evaluation metrics as well to tell how well your learning algorithm is doing at finding that small handful of anomalies or positive examples amidst this much larger set of negative examples of normal plane engines. If you didn't watch that video, don't worry about it. It's okay. The intuition I hope you get is to use the cross-validation set to just look at how many anomalies is finding and also how many normal engines is incorrectly flagging as an anomaly. Then to just use that to try to choose a good choice for the parameter Epsilon.

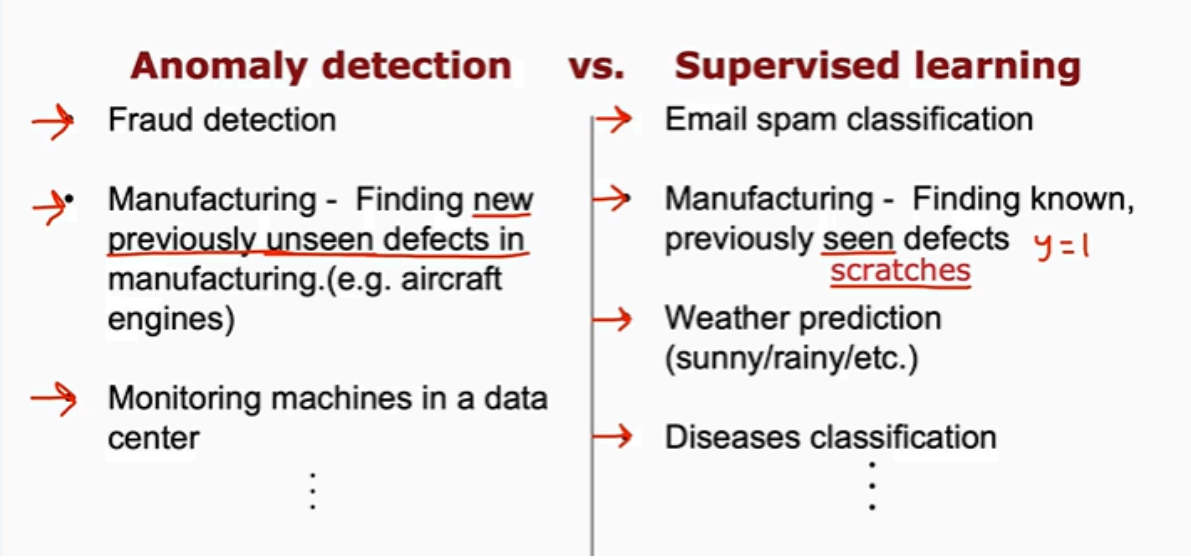
Anomaly detection vs. supervised learning

When to use anomaly detection ans supervised learning

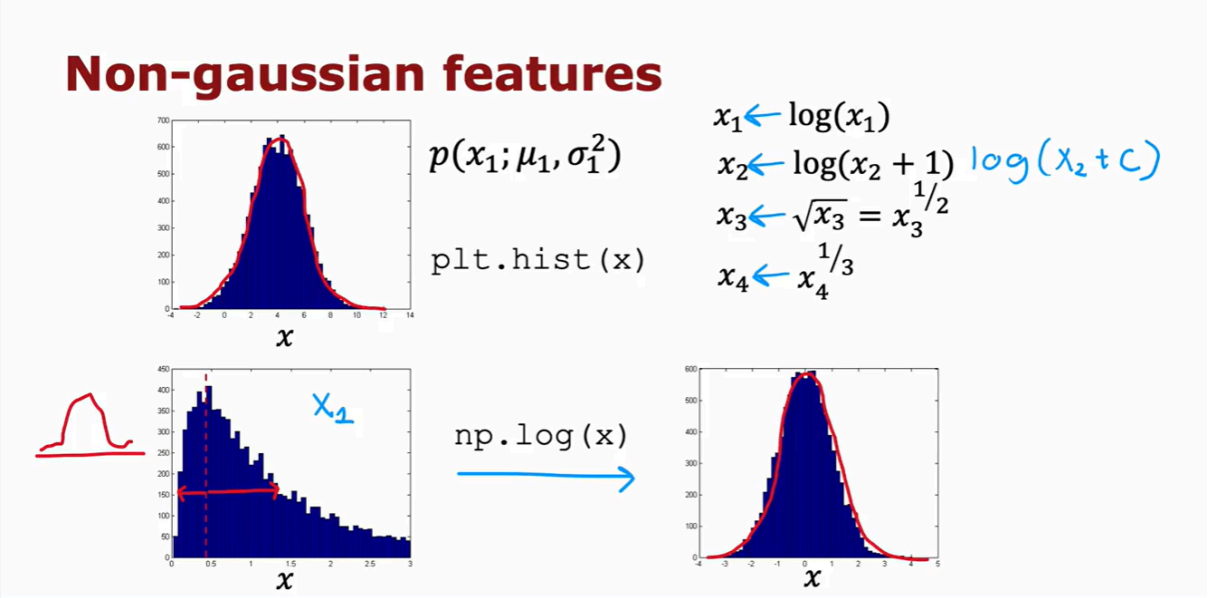


if you are using a system to find, say financial fraud. There are many different ways unfortunately that some individuals are trying to commit financial fraud. And unfortunately there are new types of financial fraud attempts every few months or every year. And what that means is that because they keep on popping up completely new. And unique forms of financial fraud anomaly detection is often used to just look for anything that's different, then transactions we've seen in the past.

In contrast, if you look at the problem of email spam detection, well, there are many different types of spam email, but even over many years. Spam emails keep on trying to sell similar things or get you to go to similar websites and so on. Spam email that you will get in the next few days is much more likely to be similar to spam emails that you have seen in the past. So that's why supervised learning works well for spam because it's trying to detect more of the types of spam emails that you have probably seen in the past in your training set. Whereas if you're trying to detect brand new types of fraud that have never been seen before, then anomaly detection maybe more applicable.



Choosing what features to use



In supervised learning, if you don't have the features quite right, or if you have a few extra features that are not relevant to the problem, that often turns out to be okay. Because the algorithm has to supervised signal that is enough labels why for the algorithm to figure out what features ignore, or how to re scale feature and to take the best advantage of the features you do give it.

But for anomaly detection which runs, or learns just from unlabeled data, is harder for the algorithm to figure out what features to ignore. So I found that carefully choosing the features, is even more important for anomaly detection, than for supervised learning approaches. Let's take a look at this video as

some practical tips, for how to tune the features for anomaly detection, to try to get you the best possible performance. One step that can help your anomaly detection algorithm, is to try to make sure the features you give it are more or less Gaussian.

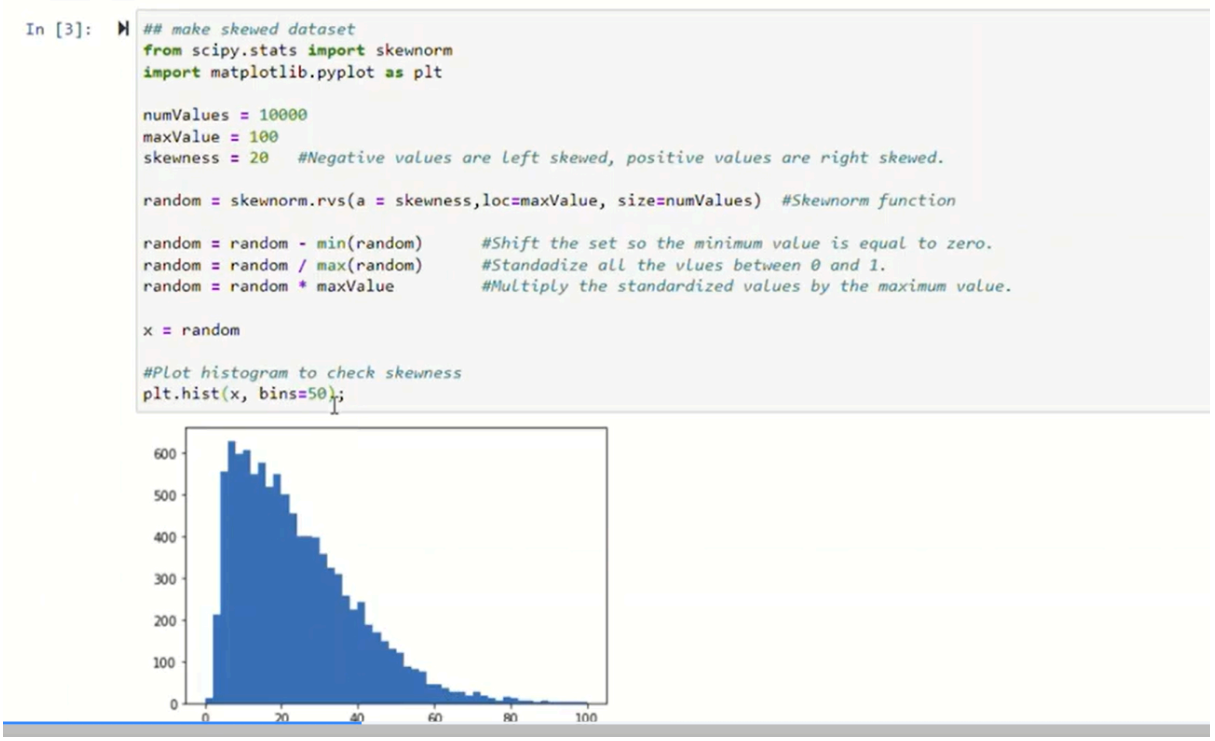
And if your features are not Gaussian, sometimes you can change it to make it a little bit more Gaussian. Let me show you what I mean. If you have a feature X, I will often plot a histogram of the feature which you can do using the python command PLT. Though you see this in the practice lab as well, in order to look at the histogram of the data. This distribution here (the diagram in the top) looks pretty Gaussian. So this would be a good candidate feature. If you think this is a feature that helps distinguish between anomalies and normal examples.

But quite often when you plot a histogram of your features, you may find that the feature has a distribution like this (the diagram in the bottom). This does not at all look like that symmetric bell shaped curve. When that is the case, I would consider if you can take this feature X, and transform it in order to make a more Gaussian.

- For example, maybe if you were to compute the log of X and plot a histogram of log(X), look like this (the diagram in the right), and this looks much more Gaussian. And so if this feature was feature x_1, then instead of using the original feature x_1 which looks like this on the left, you might instead replace that feature with log(x_1), to get this distribution over here (the diagram in the right). Because when x_1 is made more Gaussian. When anomaly detection models P(x_1) using a Gaussian distribution like that, is more likely to be a good fit to the data. Other than the log function
- Other things you might do is, given a different feature x_2, you may replace it with x_1, log($x_2 + 1$). This would be a different way of transforming x_2. And more generally, log($x_2 + c$), would be one example of a formula you can use, to change X to try to make it more Gaussian.
- Or for a different feature, you might try taking the square root or really the square would have executed this X lead to the power of one half,and you may change that exponentially term. So for a different feature X four, you might use X four to the power of one third, for example.

So when I'm building an anomaly detection system, I'll sometimes take a look at my features, and if I see any highly non Gaussian by plotting histogram, I might choose transformations like these or others, In order to try to make it more Gaussian. It turns out a larger value of C, will end up transforming this

distribution less. But in practice I just try a bunch of different values of C, and then try to take a look to pick one that looks better in terms of making the distribution more Gaussian.

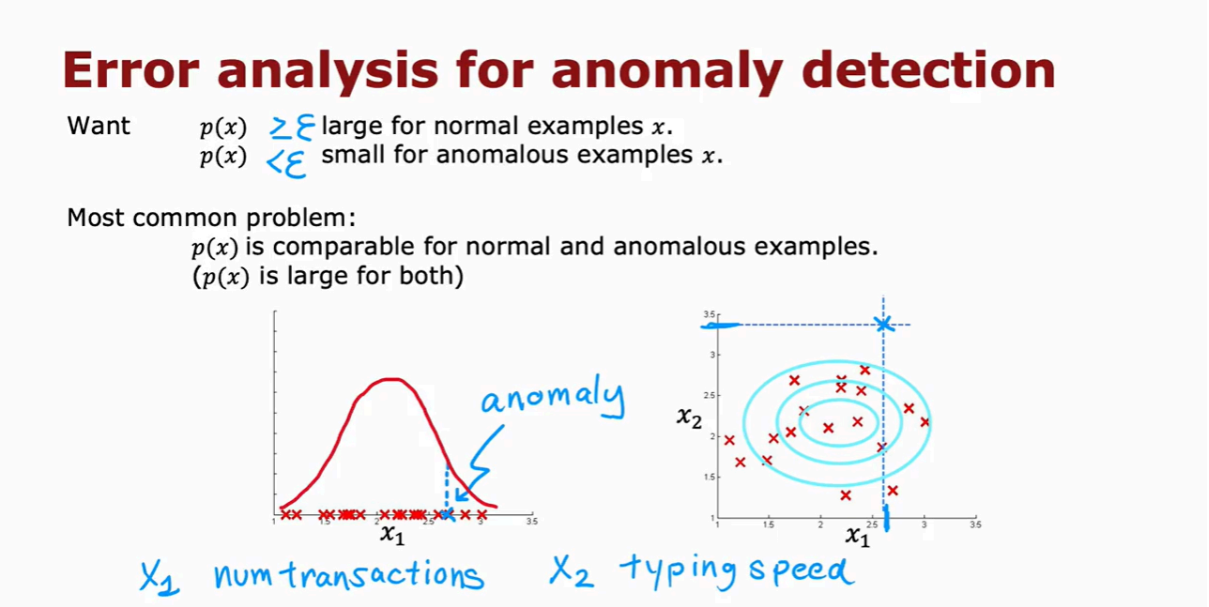


And just as I'm doing right now in real time, you can see that, you can very quickly change these parameters and plot the histogram. In order to try to take

a look and try to get something a bit more Gaussian, than was the original data next that you saw in this histogram up above. If you read the machine learning literature, there are some ways to automatically measure how close these distributions are to Gaussian. But I found it in practice, it doesn't make a big difference, if you just try a few values and pick something that looks right to you, that will work well for all practical purposes. So, by trying things out in Jupiter notebook, you can try to pick a transformation that makes your data more Gaussian.

And just as a reminder, whatever transformation you apply to the training set, please remember to apply the same transformation to your cross validation and test set data as well

Error analysis for anomaly detection



When you've learned the model P(x) from your unlabeled data, the most common problem that you may run into is that, P(X) is comparable in value say is, large for both normal and for anomalous examples. As a concrete example, if this is your data set, you might fit that Gaussian into it.

And if you have an example in your cross validation set or test set, that is over here (the diagram in the left), that is anomalous, then this has a pretty high probability. And in fact, it looks quite similar to the other examples in your training set. And so, even though this is an anomaly, P(X) is actually pretty large. And so the algorithm will fail to flag this particular example as an anomaly. In that case, what I would normally do is, try to look at that example

and try to figure out what is it that made me think is an anomaly, even if this feature X_1 took on values similar to other training examples. And if I can identify some new feature say X_2, that helps distinguish this example from the normal examples.

Then adding that feature, can help improve the performance of the algorithm. Here's a picture showing what I mean. If I can come up with a new feature X_2, say, I'm trying to detect fraudulent behavior, and if X_1 is the number of transactions they make, maybe this user looks like they're making some of the transactions as everyone else. But if I discover that this user has some insanely fast typing speed, and if I were to add a new feature X_2, that is the typing speed of this user. And if it turns out that when I plot this data using the old feature X_1 and this new feature X_2, causes X two to stand out over here.

Then it becomes much easier for the anomaly detection algorithm to recognize an X_2 is an anomalous user. Because when you have this new feature X_2, the learning algorithm may fit a Gaussian distribution that assigns high probability to points in this region, a bit lower in this region, and a bit lower in this region. And so this example, because of the very anomalous value of X_2, becomes easier to detect as an anomaly. So just to summarize the development process will often go through is, to train the model and then to see what anomalies in the cross validation set the algorithm is failing to detect. And then to look at those examples to see if that can inspire the creation of new features that would allow the algorithm to spot. That example takes on unusually large or unusually small values on the new features, so that you can now successfully flag those examples as anomalies.